Home Assignment 2

Roque Hauser

2026-11-03

Data cleaning

First, the dataset HA2data_2022.xlsx was cleaned and split into three seperate datasets: cleaned daily data, cleaned monthly data and cleaned monthly us cpi.

R Code

Cleaning of daily data

```
# Daily cleaning code

# Import dataset daily
library(readxl)

daily_excel <- read_excel("01_data/raw/HA2data_2022.xlsx",
                          sheet = "Daily")


# Cleaned dataset daily
library(dplyr)

daily_cleaned <- daily_excel %>%
  select(-c(39, 42)) %>%
  select(where(~ !all(is.na(.)))) %>%
  setNames(make.names(as.character(unlist(.[1, ])),
                      unique = TRUE)) %>%
  slice(-1) %>%
  mutate(across(everything(), type.convert, as.is = TRUE)) %>%
  rename(Date = 1) %>%
  mutate(Date = as.Date(Date, origin = "1899-12-30"))

write.csv(daily_cleaned,
          "01_data/cleaned/data_cleaned_daily_HA2-6.csv",
          row.names = FALSE)
```

Cleaning of monthly data

```r
# Monthly cleaning code

# Import dataset monthly
library(readxl)

monthly_excel <- read_excel("01_data/raw/HA2data_2022.xlsx",
                            sheet = "Monthly")

# Cleaned dataset monthly
library(dplyr)

monthly_cleaned <- monthly_excel %>%
  select(-c(39, 42)) %>%
  select(where(~ !all(is.na(.)))) %>%
  setNames(make.names(as.character(unlist(.[1, ])),
                      unique = TRUE)) %>%
  slice(-1) %>%
  mutate(across(everything(), type.convert, as.is = TRUE)) %>%
  rename(Date = 1) %>%
  mutate(Date = as.Date(Date, origin = "1899-12-30"))

write.csv(monthly_cleaned,
          "01_data/cleaned/data_cleaned_monthly_HA2-6.csv",
          row.names = FALSE)
```

Cleaning of monthly US CPI data

```
# CPI cleaning code

# Import dataset US CPI index
library(readxl)

us_cpi_excel <- read_excel("01_data/raw/HA2data_2022.xlsx",
                           sheet = "US CPI index")


# Cleaned dataset US CPI index
library(dplyr)
library(tidyr)

us_cpi_cleaned <- us_cpi_excel %>%
  rename(Year = 1) %>%
  select(-Average) %>%
  pivot_longer(
    cols = -Year,
    names_to = "Month",
    values_to = "CPI"
  ) %>%
  mutate(
    Year = as.integer(Year),
    Month = as.integer(Month),
    Date = as.Date(sprintf("%04d-%02d-01", Year, Month))
  ) %>%
  filter(!is.na(CPI)) %>%
  select(Date, CPI) %>%
  arrange(Date)


write.csv(us_cpi_cleaned,
          "01_data/cleaned/data_cleaned_us_cpi_HA2-6.csv",
          row.names = FALSE)
```

Exercise 1

Question

Use R to calculate descriptive statistics for the five stocks' simple daily returns. Test each series for normality with the Jarque–Bera test. For how many series can you reject the null hypothesis of normality?

Answer

The Jarque–Bera tests reject the null hypothesis of normally distributed daily returns for all five stocks at the 5% significance level ($p < 0.05$). Therefore, the correct answer is **d. More than 3**.

Table 1: Jarque-Bera Test Results

Stock	JB Statistic	p-value
Apple	4071.86	<2e-16
Microsoft	10020.97	<2e-16
Tesla	4153.33	<2e-16
JP Morgan Chase	15489.17	<2e-16
Walmart	36466.84	<2e-16

R Code

```
# Exercise 1

# Filter daily data to include only observations up to the end of 2022
daily_data_2022 <- daily_cleaned %>%
  filter(!is.na(Date), Date <= as.Date("2022-12-31")) %>%
  arrange(Date)

stocks <- c(
  "APPLE",
  "MICROSOFT",
  "TESLA",
  "JP.MORGAN.CHASE...CO.",
  "WALMART"
)

stock_names <- c(
  "apple",
  "microsoft",
  "tesla",
  "jp_morgan_chase",
  "walmart"
)


# Calculate simple daily returns
for (i in seq_along(stocks)) {
```

```r
  daily_returns <- daily_data_2022 %>%
    mutate(Return = .data[[stocks[i]]] / lag(.data[[stocks[i]]]) - 1) %>%
    select(Date, Return)

  assign(paste0(stock_names[i], "_daily_returns"), daily_returns)
}

# Jarque-Bera test
library(tseries)

jb_apple <- jarque.bera.test(na.omit(apple_daily_returns$Return))
jb_microsoft <- jarque.bera.test(na.omit(microsoft_daily_returns$Return))
jb_tesla <- jarque.bera.test(na.omit(tesla_daily_returns$Return))
jb_jp_morgan <- jarque.bera.test(na.omit(jp_morgan_chase_daily_returns$Return))
jb_walmart <- jarque.bera.test(na.omit(walmart_daily_returns$Return))

# Display test statistics and p-values
jb_results <- data.frame(
  Stock = c("Apple", "Microsoft", "Tesla", "JP Morgan Chase", "Walmart"),
  JB_Statistic = round(c(
    jb_apple$statistic,
    jb_microsoft$statistic,
    jb_tesla$statistic,
    jb_jp_morgan$statistic,
    jb_walmart$statistic
  ), 2),
  P_Value = format.pval(c(
    jb_apple$p.value,
    jb_microsoft$p.value,
    jb_tesla$p.value,
    jb_jp_morgan$p.value,
    jb_walmart$p.value
  ), digits = 3, eps = 2.2e-16)
)

print(jb_results, row.names = FALSE)
```

Exercise 2

Question

Which stock had the lowest average daily bid–ask spread (%) during the sample?

Answer

Apple has the lowest average daily bid–ask spread (%). Therefore, the correct answer is **a. Apple**

Table 2: Average Daily Bid-Ask Spreads

Stock	Average Spread (%)
apple	0.0174
microsoft	0.0245
tesla	0.0448
jp_morgan_chase	0.0261
walmart	0.0283

R Code

```r
# Exercise 2

# Calculate average daily bid-ask spreads in percent
avg_spreads <- numeric(length(stocks))

for (i in seq_along(stocks)) {

  ask <- daily_data_2022[[paste0(stocks[i], "...ASK.PRICE")]]
  bid <- daily_data_2022[[paste0(stocks[i], "...BID.PRICE")]]

  spread <- (ask - bid) / ((ask + bid) / 2) * 100

  avg_spreads[i] <- mean(spread, na.rm = TRUE)
}

# Display results
spread_results <- data.frame(
  Stock = stock_names,
  Average_Spread_Percent = round(avg_spreads, 4)
)

print(spread_results)
```

Exercise 3

Question

Use monthly simple returns for this and all remaining questions. If you invest 100% in the stock with the highest average return, what is your average monthly return? What is the average monthly return if you instead select the worst-performing stock? Answer: What is the difference between the best and worst average rates of return per annum?

Answer

The difference between the highest and lowest annualized average returns is 50.27 percentage points. Therefore, the correct answer is **d. More than 15 percentage points**

Table 3: Average Monthly and Annualized Returns

Stock	Monthly Return (%)	Annualized Return (%)
APPLE	2.09	25.06
MICROSOFT	1.69	20.32
TESLA	4.94	59.33
JP.MORGAN.CHASE...CO.	0.91	10.90
WALMART	0.76	9.06

R Code

```
# Exercise 3

# Filter monthly data to include only observations up to the end of 2022
monthly_data_2022 <- monthly_cleaned %>%
  filter(!is.na(Date), Date <= as.Date("2022-12-31")) %>%
  arrange(Date)

# Calculate simple monthly returns
for (i in seq_along(stocks)) {

  monthly_returns <- monthly_data_2022 %>%
    mutate(Return = .data[[stocks[i]]] / lag(.data[[stocks[i]]]) - 1) %>%
    select(Date, Return)

  assign(paste0(stock_names[i], "_monthly_returns"), monthly_returns)
}

# Calculate average monthly and annual returns
avg_monthly_returns <- numeric(length(stocks))

for (i in seq_along(stocks)) {

  help_returns <- get(paste0(stock_names[i], "_monthly_returns"))$Return

  avg_monthly_returns[i] <- mean(help_returns, na.rm = TRUE)
```

```r
}

# Create results table
return_results <- data.frame(
  Stock = stocks,
  Monthly_Return = avg_monthly_returns * 100,
  Annual_Return = avg_monthly_returns * 12 * 100
)

# Calculate annual return difference in percentage points
return_difference <- max(return_results$Annual_Return) -
  min(return_results$Annual_Return)

print(return_difference)
```

Exercise 4

Question

Assume that you can predict each month's best-performing stock and invest 100% in it each month. What is your rate of return per annum? If you instead select the worst-performing stock every month, what is your average monthly return? Answer: What is the difference between the best and worst rates of return per annum?

Answer

The difference between the best and worst annualized returns is 223.79 percentage points. Therefore, the correct answer is **d. More than 15 percentage points**

Table 4: Best and Worst Monthly Stock Selection

Strategy	Monthly Return (%)	Annualized Return (%)
Best Stock	12.28	147.34
Worst Stock	-6.37	-76.46

R Code

```
# Exercise 4

# Combine monthly returns of all five stocks
monthly_returns_combined <- data.frame(
  Date = apple_monthly_returns$Date,
  Apple = apple_monthly_returns$Return,
  Microsoft = microsoft_monthly_returns$Return,
  Tesla = tesla_monthly_returns$Return,
  JP_Morgan = jp_morgan_chase_monthly_returns$Return,
  Walmart = walmart_monthly_returns$Return
)

# Remove months with missing returns
monthly_returns_combined <- na.omit(monthly_returns_combined)

# Select the best and worst stock return each month
best_monthly_returns <- apply(
  monthly_returns_combined[, -1], 1, max
)

worst_monthly_returns <- apply(
  monthly_returns_combined[, -1], 1, min
)

# Calculate average monthly and annualized returns
exercise_4_results <- data.frame(
  Strategy = c("Best Stock", "Worst Stock"),
  Monthly_Return = c(
```

```
    mean(best_monthly_returns),
    mean(worst_monthly_returns)
  ) * 100,
  Annual_Return = c(
    mean(best_monthly_returns),
    mean(worst_monthly_returns)
  ) * 12 * 100
)

# Calculate annual return difference in percentage points
return_difference_4 <- diff(
  rev(exercise_4_results$Annual_Return)
)

# Display results
print(exercise_4_results)
print(return_difference_4)
```

Exercise 5

Question

Calculate the simple return on an equally weighted portfolio of all five stocks. Keep the portfolio equally weighted. What is its mean return per annum?

Answer

The equally weighted portfolio has an average annualized return of 24.93%, assuming monthly rebalancing. Therefore, the correct answer is **d. More than 15%**

Table 5: Equally Weighted Portfolio Returns

Portfolio	Monthly Return (%)	Annualized Return (%)
Equally Weighted	2.08	24.93

R Code

```
# Exercise 5

# Calculate equally weighted portfolio returns (20% per stock)
equal_weighted_returns <- monthly_returns_combined %>%
  mutate(Portfolio_Return = rowMeans(select(., -Date)))

# Calculate average monthly and annualized portfolio returns
equal_weighted_monthly_mean <- mean(equal_weighted_returns$Portfolio_Return)

equal_weighted_annual_mean <- equal_weighted_monthly_mean * 12 * 100

# Display result
print(equal_weighted_annual_mean)
```

Exercise 6

Question

Calculate the simple return on a value-weighted portfolio using market capitalisation weights. What is its mean return per annum?

Answer

The value-weighted portfolio has an average annualized return of 19.66%. Therefore, the correct answer is **d. More than 15%**

R Code

```
# Exercise 6

# Combine monthly returns and market capitalizations
returns_matrix <- monthly_data_2022 %>%
  select(all_of(stocks)) %>%
  mutate(across(
    everything(),
    ~ .x / lag(.x) - 1
  ))

market_caps <- monthly_data_2022 %>%
  select(all_of(paste0(stocks, "...MARKET.VALUE")))

# Calculate portfolio weights using previous month's market capitalization
market_caps_lagged <- market_caps %>%
  mutate(across(everything(), ~ lag(.x)))

# Calculate monthly value-weighted portfolio returns
value_weighted_returns <- numeric(nrow(monthly_data_2022))

for (i in seq_len(nrow(monthly_data_2022))) {

  help_6_returns <- as.numeric(returns_matrix[i, ])
  caps <- as.numeric(market_caps_lagged[i, ])

  if (all(is.finite(help_6_returns)) &&
      all(is.finite(caps)) &&
      all(caps > 0)) {

    weights <- caps / sum(caps)
    value_weighted_returns[i] <- sum(weights * help_6_returns)

  } else {
    value_weighted_returns[i] <- NA_real_
  }
}
```

```r
# Calculate average monthly and annualized returns
value_weighted_monthly_mean <- mean(value_weighted_returns, na.rm = TRUE)
value_weighted_annual_mean <- value_weighted_monthly_mean * 12 * 100

# Display result
print(value_weighted_annual_mean)
```

Exercise 7

Question

Invest the same amount in each stock at the beginning of the period and then make no further trades. Calculate the portfolio's simple returns. What is its mean return per annum?

Answer

The buy-and-hold portfolio starts with an equal allocation of 20% to each stock. Unlike an equally weighted portfolio, no rebalancing is performed, so the portfolio weights change over time as stock prices fluctuate.

The average annualized return of the buy-and-hold portfolio is 37.12%.

Therefore, the correct answer is **d. More than 15%**

R Code

```r
# Exercise 7

# Select monthly stock prices
prices <- monthly_data_2022 %>%
  select(Date, all_of(stocks)) %>%
  filter(if_all(all_of(stocks), ~ !is.na(.x) & .x > 0)) %>%
  arrange(Date)

# Invest 20% in each stock at the beginning of the period
initial_weights <- rep(0.2, length(stocks))

# Calculate the number of shares purchased initially
initial_prices <- as.numeric(prices[1, stocks])
shares <- initial_weights / initial_prices

# Calculate portfolio value over time without rebalancing
portfolio_values <- as.matrix(prices[stocks]) %*% shares

# Calculate simple monthly portfolio returns
buy_hold_returns <- data.frame(
  Date = prices$Date,
  Return = as.numeric(portfolio_values / dplyr::lag(as.numeric(portfolio_values)) - 1)
)

# Calculate average monthly and annualized returns
buy_hold_monthly_mean <- mean(buy_hold_returns$Return, na.rm = TRUE)
buy_hold_annual_mean <- buy_hold_monthly_mean * 12 * 100

# Display result
print(buy_hold_annual_mean)
```

Exercise 8

Question

A buy-and-hold portfolio involves investing at the beginning of the period and making no subsequent trades, except possibly reinvesting dividends. Which of the portfolios in Q5 and Q6 most closely resembles a buy-and-hold portfolio?

Answer

A value-weighted portfolio most closely resembles a buy-and-hold strategy because portfolio weights naturally change with stock prices. Stocks that increase in value gain a larger portfolio weight, while those that decline lose weight.

In contrast, an equally weighted portfolio requires regular rebalancing to maintain equal allocations. Therefore, a value-weighted portfolio is more consistent with a buy-and-hold strategy.

Therefore, the correct answer is **b. Value-weighted portfolio**

Exercise 9

Question

What is the sum of the stock weights when the target return is 1% per month?

R hint: use solve() for linear systems and %*% for matrix multiplication. The risky-asset weights form a 5 $\times$ 1 vector; the remaining weight is invested in the risk-free asset.

(For the complete question, please refer to the assignment.)

Answer

Using historical monthly returns, the sum of the five risky-asset weights at a target return of 1% per month is **0.35153**. This corresponds to an allocation of approximately 35.15% to risky assets and 64.85% to the risk-free asset.

The portfolio volatility is approximately 2.44% per month. Both methods of calculating volatility produce identical results up to numerical precision, confirming the consistency of the calculations.

Therefore, the correct answer is **b. 0.2 < SUM < 0.4**

Table 6: Capital Allocation Line Results

Target Return (%)	Sum of Stock Weights	Volatility (%)	Direct Volatility (%)
0.0	-0.0879	0.6092	0.6092
0.1	-0.0439	0.3046	0.3046
0.2	0.0000	0.0000	0.0000
0.3	0.0439	0.3046	0.3046
0.4	0.0879	0.6092	0.6092
0.5	0.1318	0.9137	0.9137
0.6	0.1758	1.2183	1.2183
0.7	0.2197	1.5229	1.5229
0.8	0.2636	1.8275	1.8275
0.9	0.3076	2.1321	2.1321
1.0	0.3515	2.4366	2.4366
1.1	0.3955	2.7412	2.7412
1.2	0.4394	3.0458	3.0458
1.3	0.4834	3.3504	3.3504
1.4	0.5273	3.6549	3.6549
1.5	0.5712	3.9595	3.9595
1.6	0.6152	4.2641	4.2641
1.7	0.6591	4.5687	4.5687
1.8	0.7031	4.8733	4.8733
1.9	0.7470	5.1778	5.1778
2.0	0.7909	5.4824	5.4824

R Code

```
# Exercise 9: Capital Allocation Line

# Monthly returns of all five stocks
```

```r
monthly_returns_matrix <- as.matrix(
  monthly_returns_combined[, -1]
)

# Mean-return vector and sample covariance matrix
R <- colMeans(monthly_returns_matrix)
Sigma <- cov(monthly_returns_matrix)

# Risk-free rate and vector of ones
Rf <- 0.002
ones <- rep(1, length(R))

# Calculate A, B, C and K squared
Sigma_inv_R <- solve(Sigma, R)
Sigma_inv_ones <- solve(Sigma, ones)

A <- as.numeric(t(R) %*% Sigma_inv_R)
B <- as.numeric(t(ones) %*% Sigma_inv_R)
C <- as.numeric(t(ones) %*% Sigma_inv_ones)

K2 <- A - 2 * B * Rf + C * Rf^2

# Target monthly returns: 0% to 2% in steps of 0.1%
target_returns <- seq(0, 0.02, by = 0.001)

# Calculate CAL weights and volatilities
cal_results <- data.frame()

for (mu_p in target_returns) {

  # Weights of the five risky assets
  weights <- as.numeric(
    solve(Sigma, R - Rf * ones) *
      (mu_p - Rf) / K2
  )

  # Volatility calculated from portfolio weights
  variance_weights <- as.numeric(
    t(weights) %*% Sigma %*% weights
  )

  # Volatility calculated using the direct formula
  variance_direct <- (mu_p - Rf)^2 / K2

  # Store results
  cal_results <- rbind(cal_results, data.frame(
    Target_Return = mu_p * 100,
    Apple = weights[1],
    Microsoft = weights[2],
    Tesla = weights[3],
    JP_Morgan = weights[4],
    Walmart = weights[5],
    Sum_Weights = sum(weights),
```

```
    Risk_Free_Weight = 1 - sum(weights),
    Volatility_Weights = sqrt(variance_weights) * 100,
    Volatility_Direct = sqrt(variance_direct) * 100
  ))
}

# Check whether both volatility calculations agree
volatility_difference <- max(abs(
  cal_results$Volatility_Weights -
    cal_results$Volatility_Direct
))

# Results for a target monthly return of 1%
cal_result_1pct <- cal_results[
  which.min(abs(cal_results$Target_Return - 1)),
]

cal_weight_sum <- cal_result_1pct$Sum_Weights

# Display results
print(cal_results)
print(cal_result_1pct)
print(cal_weight_sum)
print(volatility_difference)
```

Exercise 10

Question

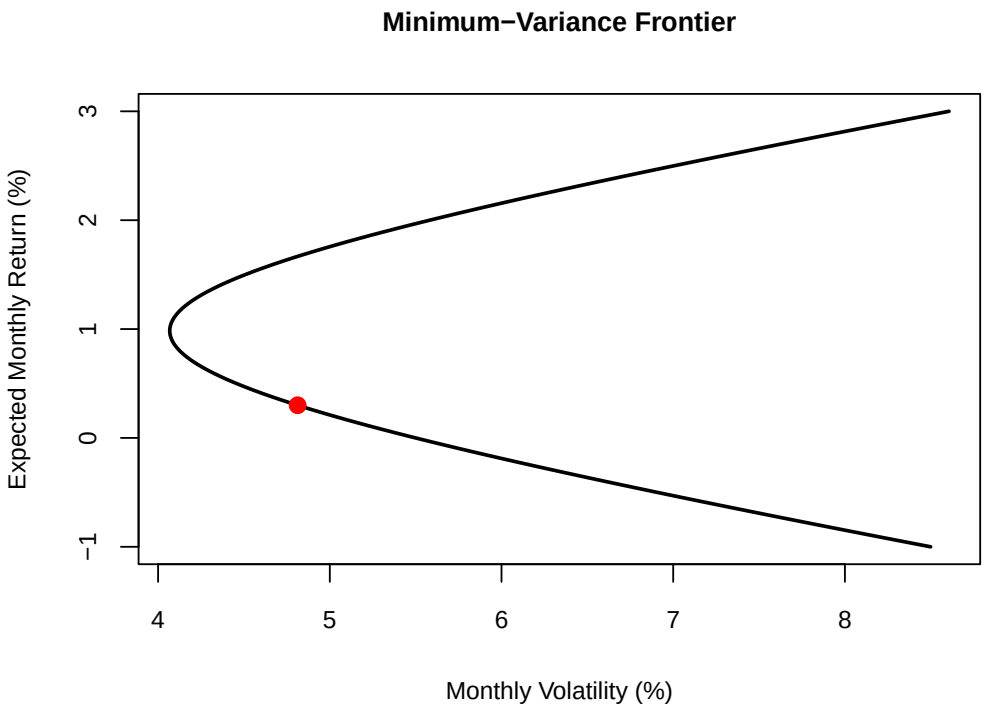
Plot the minimum-variance frontier of the five risky assets in R, allowing short selling. What is the volatility of the minimum-variance portfolio for a target return of 0.30% per month?

Answer

The minimum-variance frontier was calculated using the monthly mean-return vector and sample variance-covariance matrix of the five stocks, allowing short selling.

For a target monthly return of 0.30%, the minimum-variance portfolio has a volatility of 4.81%. The portfolio weights sum to 1, and the expected portfolio return satisfies the target return constraint.

Therefore, the correct answer is **a. Less than 5%**



R Code

```r
# Exercise 10: Minimum-Variance Frontier

# Calculate denominator
D <- A * C - B^2

# Define target monthly returns for the frontier
target_returns_10 <- seq(-0.01, 0.03, by = 0.0001)
```

```r
# Calculate minimum-variance volatility
min_var_volatility <- sqrt(
  (C * target_returns_10^2 -
     2 * B * target_returns_10 + A) / D
)

# Target monthly return of 0.30%
target_return_10 <- 0.003

# Calculate minimum-variance portfolio weights
weights_min_var <- as.numeric(
  solve(Sigma,
        ((C * target_return_10 - B) / D) * R +
          ((A - B * target_return_10) / D) * ones)
)

# Calculate volatility using portfolio weights
min_var_variance <- as.numeric(
  t(weights_min_var) %*% Sigma %*% weights_min_var
)

min_var_volatility_03 <- sqrt(min_var_variance) * 100

# Check portfolio constraints
sum_weights_10 <- sum(weights_min_var)
portfolio_return_10 <- sum(weights_min_var * R)

# Plot minimum-variance frontier
plot(
  min_var_volatility * 100,
  target_returns_10 * 100,
  type = "l",
  lwd = 2,
  xlab = "Monthly Volatility (%)",
  ylab = "Expected Monthly Return (%)",
  main = "Minimum-Variance Frontier"
)

# Highlight target portfolio
points(
  min_var_volatility_03,
  target_return_10 * 100,
  col = "red",
  pch = 19,
  cex = 1.3
)

# Display results
print(weights_min_var)
print(sum_weights_10)
print(portfolio_return_10)
print(min_var_volatility_03)
```